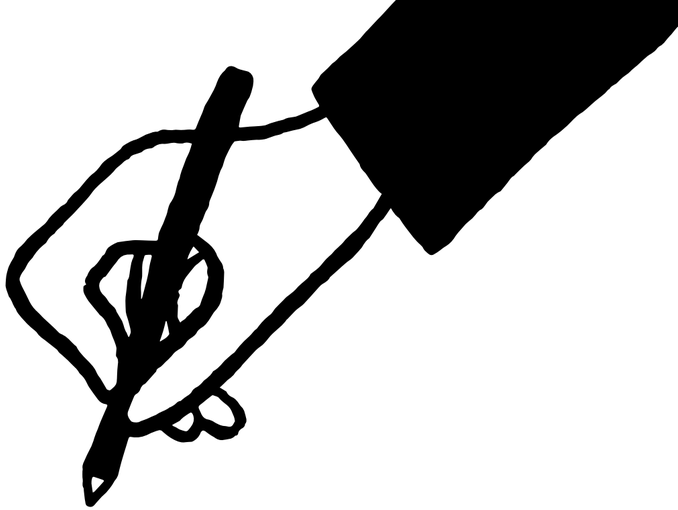


ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Vite and Vue Devtools

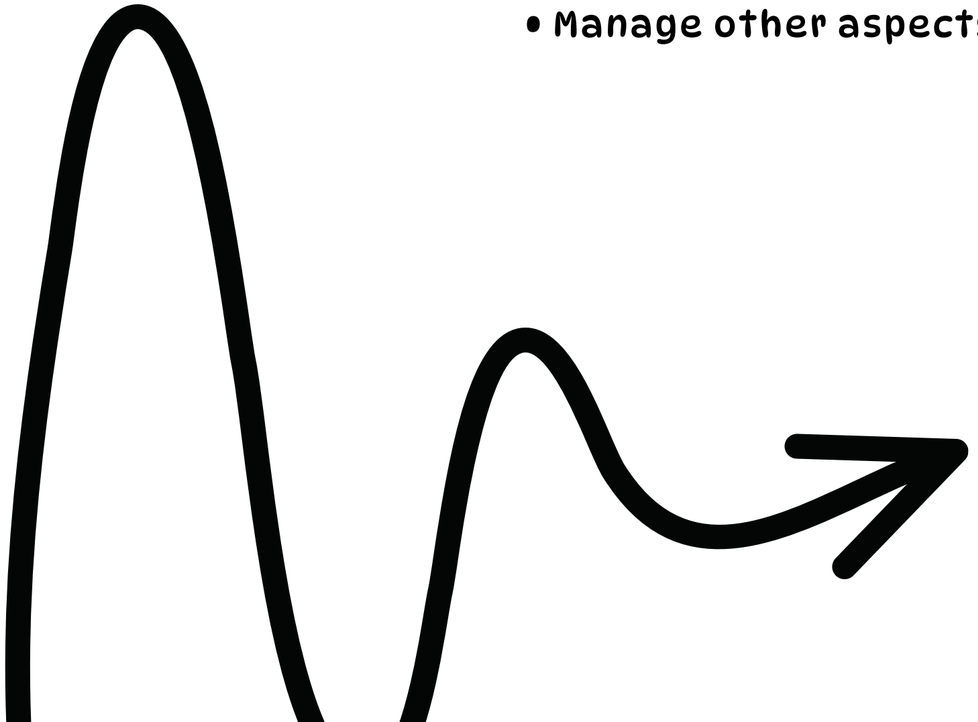
Chapter 3

Advanced Web Design

3 Vite and Vue Devtools

Vite.js is the build management tool aiming to do the following:

- Help you develop faster (locally develop your project with a more time-saving approach)
- Build with optimization (bundle files for production with better performance)
- Manage other aspects of your web project effectively (testing, linting, and so on)



3 Vite and Vue Devtools

You will need to provide additional configurations for Vite to proceed:

```
Vue.js - The Progressive JavaScript Framework

✓ Project name:   chapter-3-vue-app
✓ Add TypeScript?   No   Yes
✓ Add JSX Support?   No   Yes
✓ Add Vue Router for Single Page Application development?   No   Yes
✓ Add Pinia for state management?   No   Yes
✓ Add Vitest for Unit Testing?   No   Yes
✓ Add Cypress for End-to-End testing?   No   Yes
✓ Add ESLint for code quality?   No   Yes
✓ Add Prettier for code formatting?   No   Yes
```

- **Vue Router** for routing management (Chapter 7: Routing), and Pinia for state management (Chapter 9: The State of Vue State Management)
- **Vitest** for enabling unit testing coverage for the project
- **ESLint** for linting and **Prettier** for organizing the project code

Once done, the Vite package is also part of the dependency packages for the project. You now can run the following commands:

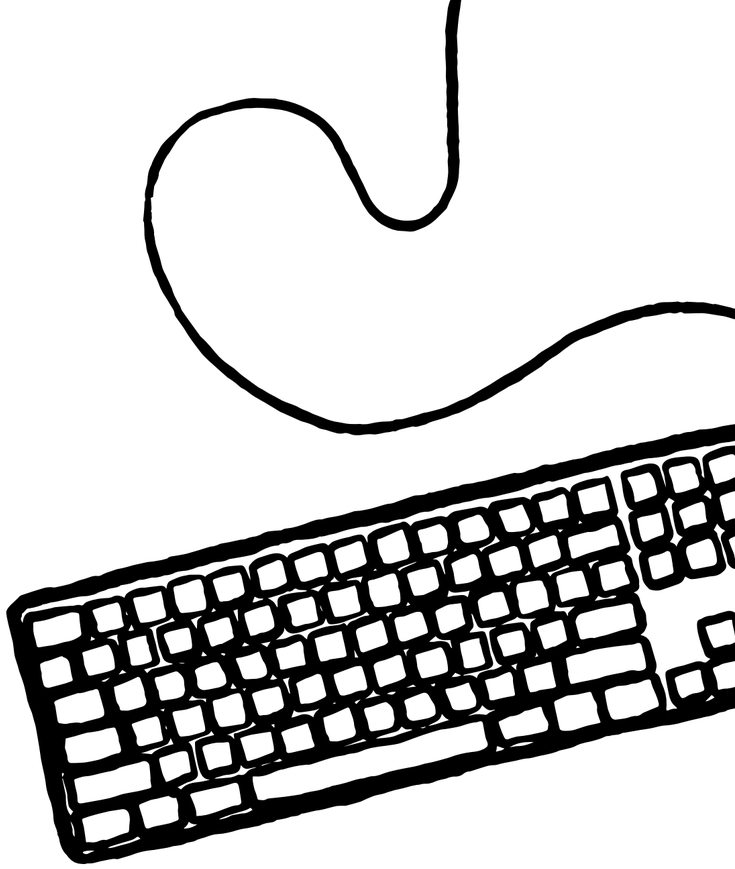
- **npm run dev or yarn dev:** Runs your project locally on localhost:3000 with hot reloading mode (HRM). Port number 3000 is arbitrarily assigned, as it is above the wellknown port numbers 1-1023 used in other areas of computing. If multiple Vue projects run simultaneously, the port number will differ between projects.

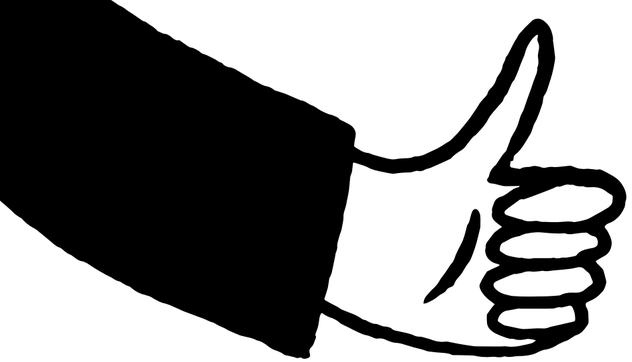
- **npm run build or yarn build:** Runs a production build that bundles your code into a single small file or several minimized small files, ready for deploying to production.

- **npm run lint or yarn lint:** Runs the process of linting, which will highlight code errors or warnings, making your code more consistent.

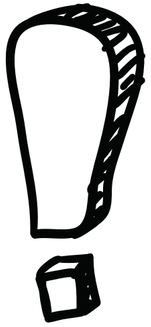
- **npm run preview or yarn preview:** Runs a preview version of the project in a specific port, simulating the production mode.

- **npm run test:unit or yarn test:unit:** Runs the project's unit tests using Vitest.



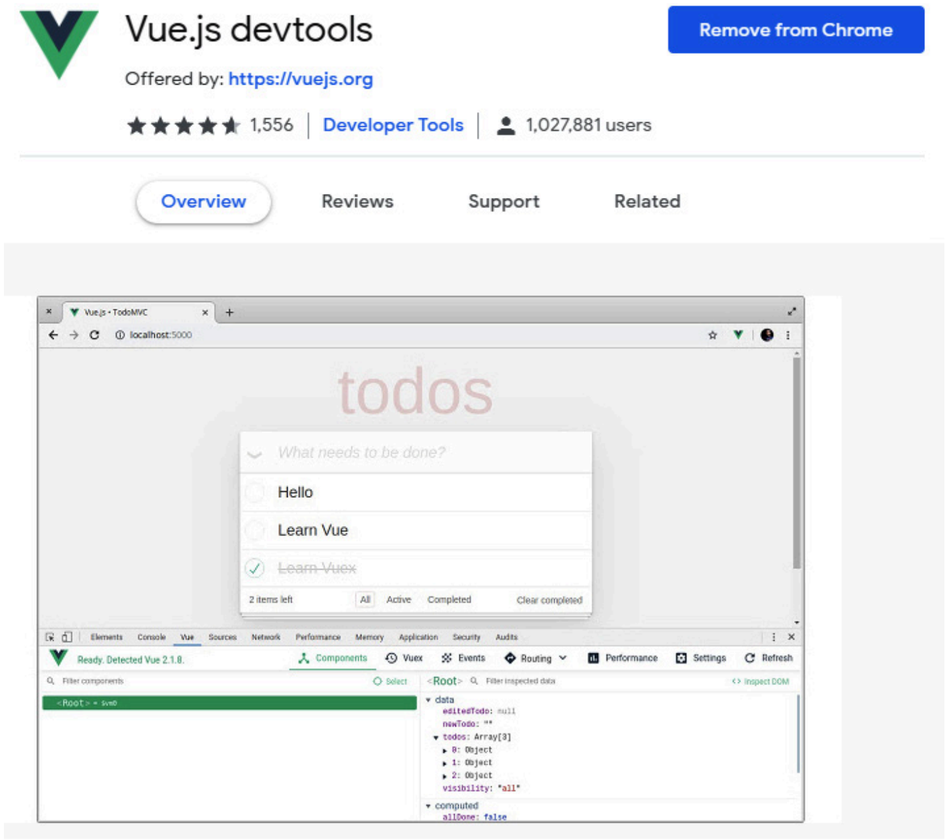


Exercise 3.01 – setting up a Vue project using Vite commands



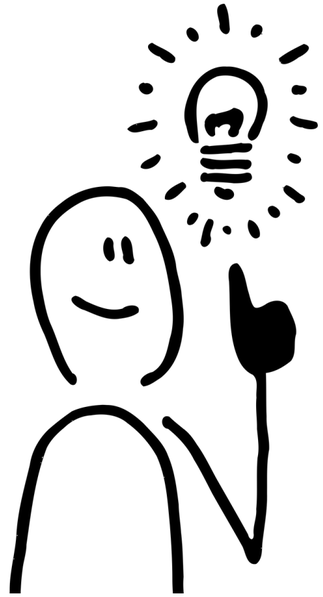
Using Vue Devtools

Vue Devtools is a browser extension for Chrome and Firefox and an Electron desktop app.
You can install and run it from your browser to debug your Vue.js projects during development. This extension does not work in production or remotely run projects.



Once enabled, the extension will add a Vue tab to the developer's console. There are two main tabs within the Vue Devtools view: **Components** and **Timeline**.

Components tab

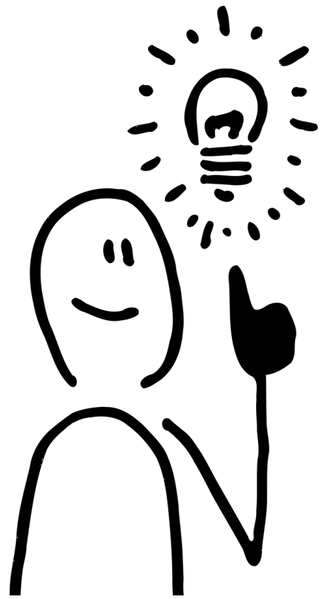


Side actions (top-right corner)

The Components tab (previously Inspector) will by default be visible once you open the Vue Devtools tab.
You can select the Vue element from the browser UI using the Select component in a page icon (top-right corner).

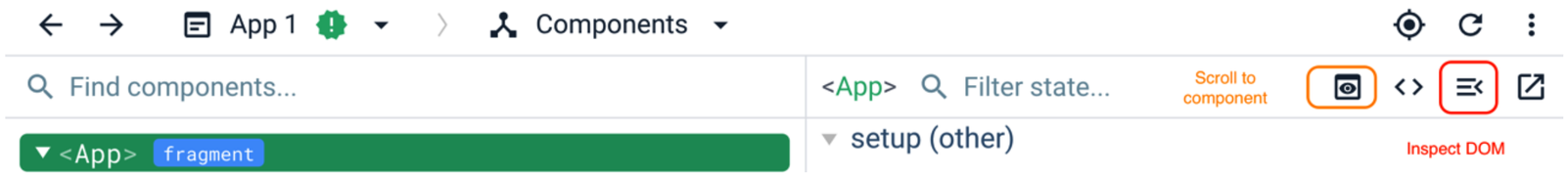
The second shortcut action is Refresh, which allows you to refresh the Devtools instance in the browser

Components tab



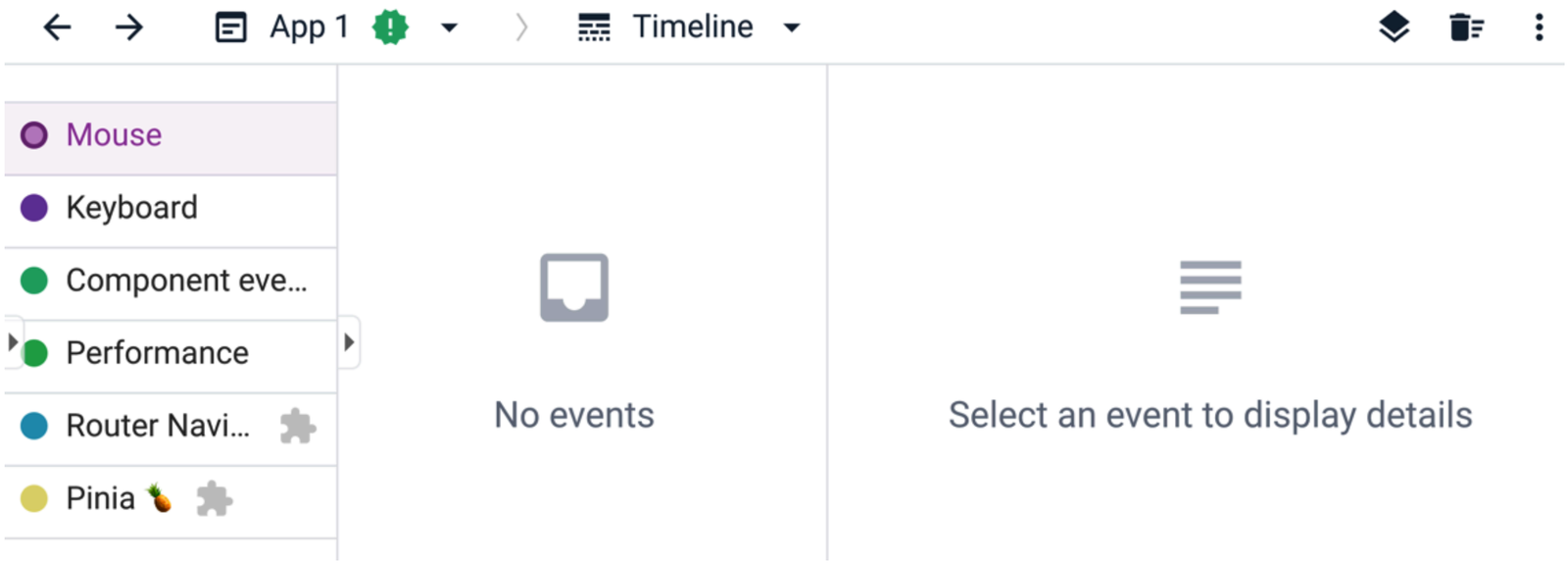
Side actions (top-right corner)

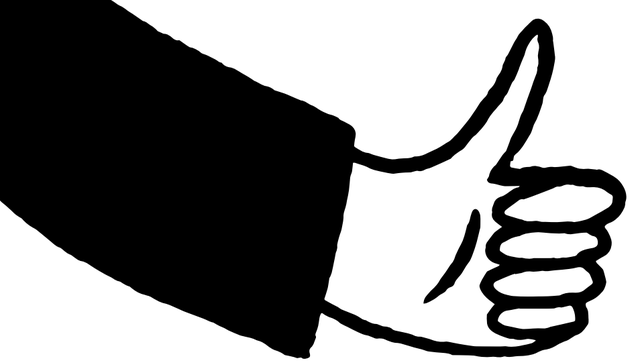
When the Components tab is active, a tree of components within the app will be available on the left-side panel. The right-side panel will display the local state and details of any selected component from the tree.
There are small shortcut actions such as Inspect DOM, which takes you directly to the location of the highlighted component in the browser's DOM tree, and Scroll to component, which will auto-scroll to the component on the UI with highlights.



Timeline tab

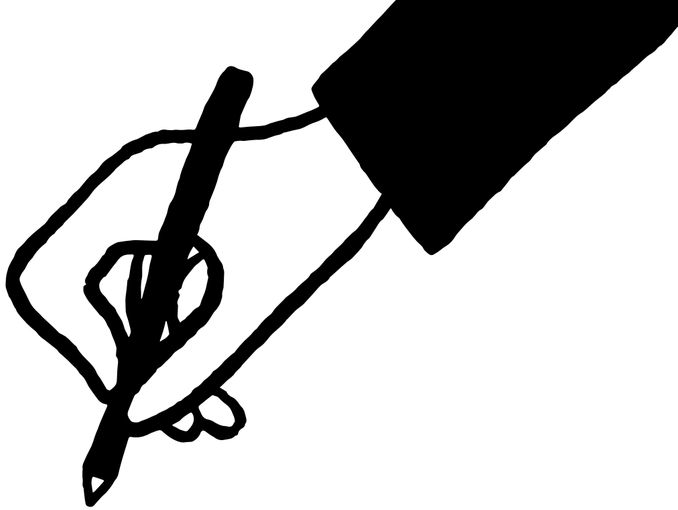
This tab records all events that happened in the app, divided into four main sections: Mouse events, Keyboard events, Component-specific events, and Performance events.





Exercise 3.02 – debugging a Vue application using Devtools

You will need to have Chrome, Firefox, or Edge installed with the Vue Devtools extension enabled.
You will use the Vue tab in the browser's developer console to inspect the code and manipulate the local data state of the component.



ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Nesting Components (Modularity)

Chapter 4

Advanced Web Design

Nesting Components (Modularity)

This chapter introduces concepts such as **props, events, prop validation, and slots.**

It also covers how to use **refs** to access DOM elements during runtime.

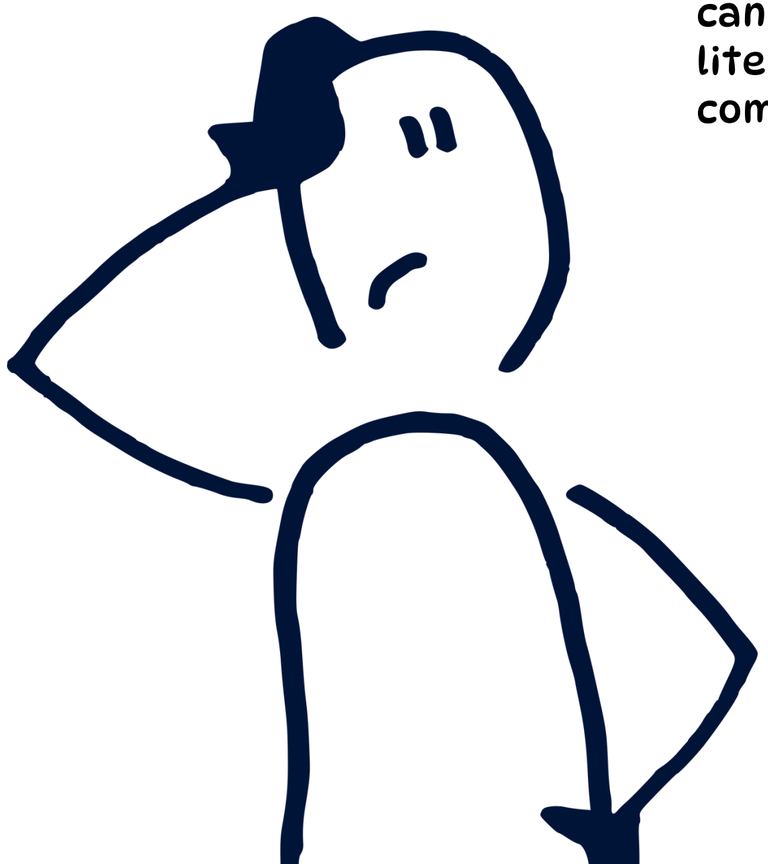
You will be able to define communication interfaces **between components** using props, events, and validators, and be ready to build components for your Vue component library or a Vue application.

This chapter covers the following topics:

- Passing props
- Understanding prop types and validation
- Understanding slots, named slots, and scoped slots
- Understanding Vue refs
- Using events for child-parent communication

Props in the context of Vue are **fields defined in a child component accessible on that component's instance (this) and in the component's template.**

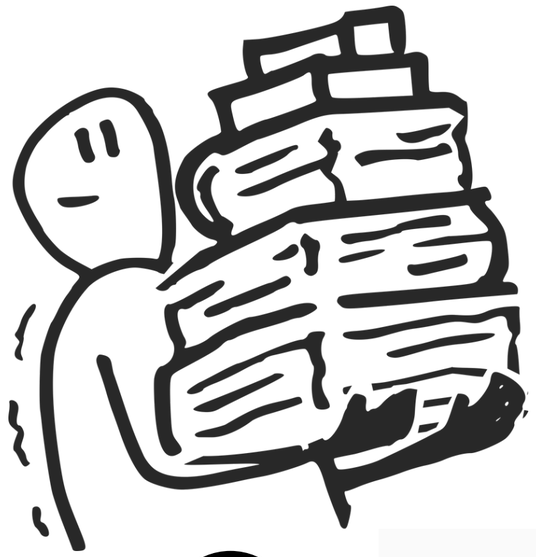
```
<template>
 <div class="hello">
   <h1>{{ msg }}</h1>
   <!-- … -->
 </div>
</template>

<script>
  export default {
   name: 'HelloWorld',
   props: ['msg']
 }
</script>
```



Passing props

* The **value** of a prop depends on what the parent passes in its template to the child component at render time.

The props property of a Vue component can be an array of strings or an object literal, each property field of which is a component's prop definition



Passing props to a component

1

```
<template>
 <div class="hello">
   <h1>{{ msg }}</h1>
   <!-- … -->
 </div>
</template>
```

2

```
<script>
  export default {
   name: 'HelloWorld',
   props: ['msg']
 }
</script>
```

3

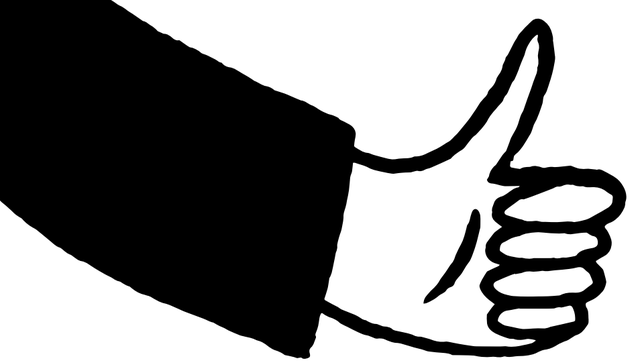
```
<script setup>
import HelloWorld from "./components/HelloWorld.vue";
</script>
```

4

```
<template>
 <div id="app">
  <HelloWorld msg="Vue.js"/>
 </div>
</template>
```

5

Hello Vue.js



Exercise 4.01 – Implementing a Greeting component

Binding reactive data to props

In the previous section, we saw how to pass static data as props to a component.
What if we want to pass reactive data from the parent to the child?
This is where binding comes in. You can use v-bind: (or : for short) to enable one-way binding of a parent's reactive data to the child component's props.

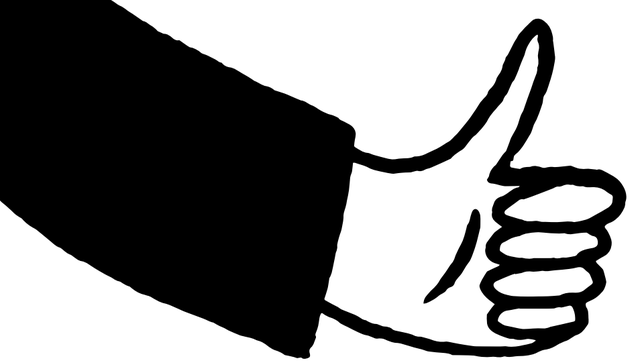
```
<template>
 <div id="app">
  <HelloWorld :msg="appWho"/>
 </div>
</template>

<script setup>
import HelloWorld from './components/HelloWorld.vue'
 const appWho = 'Vue.js'
</script>
```

Hello Vue.js

```
<template>
 <div id="app">
  <HelloWorld :msg="appWho"/>
  <button @click="setWho('JavaScript')">JavaScript</button>
  <button @click="setWho('Everyone')">Everyone</button>
 </div>
</template>

<script>
import HelloWorld from './components/HelloWorld.vue'
export default {
 components: {
  HelloWorld
 },
 data () {
  return {
  appWho: 'Vue.js'
 }
},
methods: {
 setWho(newWho) {
  this.appWho = newWho
  }
 }
}
</script>
```

Hello Vue.js

JavaScript Everyone



Exercise 4.02 – Passing reactive data that changes frequently to props

Understanding prop types and validation

Primitive prop validation

Assume we want a Repeat.vue component that takes a “times” prop and a “content” prop and then calculates the array of repetitions using “computed” based on the times value.

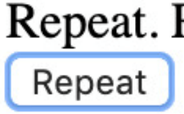
```
<template>
 <div>
  <span v-for="r in repetitions" :key="r">
   {{ content }}
  </span>
 </div>
</template>

<script>
 export default {
  props: ['times', 'content'],
  computed: {
   repetitions() {
   return Array.from({ length: this.times });
  }
 }
}
</script>
```

Understanding prop types and validation

In App.vue, we can consume our Repeat component as follows:

Output:

Whenever clicking the Repeat button, the Repeat component will display the content one more time, as follows:



```
<template>
 <div id="app">
  <Repeat :times="count" content="Repeat" />
  <button @click="increment()">Repeat</button>
 </div>
</template>

<script>
 import Repeat from './components/Repeat.vue'
 export default {
  components: {
   Repeat
  },
  data() {
   return { count: 1 }
  },
  methods: {
   increment() {
    this.count += 1
   }
  }
 }
</script>
```

Understanding prop types and validation

Now, for this component to work properly, we need times to be a Number type and, ideally, content to be a String type.

Props type in Vue can be any type including String, Number, Boolean, Array, Object, Date, Function, and Symbol.

```
<script>
 export default {
  props: {
  times: {
   type: Number
  },
  content: {
   type: String
  }
 },
// rest of component definition
}
</script>
```

Let's see what happens if we update App to pass the wrong prop type to Repeat – for example, let's say times is a String and content is a Number:

```
<template>
 <div id="app">
  <Repeat :times="count" :content="55" />
 </div>
</template>

<script>
// no changes to imports
export default {
 data() {
  return { count: 'no-number-here' }
 },
 // other properties
}
</script>
```

In this case, the Repeat component will fail to render, and the following errors will be logged to the console:

```
⚠ ▶ [Vue warn]: Invalid prop: type check failed for prop "times". Expected
Number with value NaN, got String with value "no-number-here".
    at <Repeat times="no-number-here" content=55 >
    at <App>
⚠ ▶ [Vue warn]: Invalid prop: type check failed for prop "content". Expected
String with value "55", got Number with value 55.
    at <Repeat times="no-number-here" content=55 >
    at <App>
```

- A union type is a type that is one of many other types, and is represented by an array of types, such as [String, Number]. We declare a prop to accept a union type by using the type field of that data prop object.

```
<script>
export default {
 props: {
 // other prop definitions
 content: {
  type: [String, Number]
  }
 }
// rest of component definition
}
</script>
```

- this will show no errors:

```
<template>
 <div id="app">
  <Repeat :times="3" :content="55" />
 </div>
</template>
```

Understanding union and custom prop types

- We can also use any valid JavaScript constructor as a prop's type, such as a Promise or a custom User class constructor

```
<script>
import User from './user.js'
export default {
 props: {
  todoListPromise: {
   type: Promise
  },
  currentUser: {
   type: User
  }
 }
}
</script>
```

Understanding union and custom prop types

- Note here we import the User custom type from another file. We can use this TodoList component as follows:

```
<template>
 <div>
  <div v-if="todosPromise && !error">
  <TodoList
   :todoListPromise="todosPromise"
   :currentUser="currentUser"
  />
  </div>
{{ error }}
 </div>
</template>
```

- Vue uses **instanceof** validation internally, so make sure any custom types are instantiated using the relevant constructor.
- Passing null or undefined will fail the check for Array and Object.
- Passing an array will pass the check for Object since an array is also an instance of Object in JavaScript.

```
<script>
import TodoList from './components/TodoList.vue'
import User from './components/user.js'

const currentUser = new User()
export default {
 components: {
  TodoList
 },
 mounted() {
  this.todosPromise = fetch('/api/todos').then(res => {
  if (res.ok) {
   return res.json()
  }
  throw new Error('Could not fetch todos')
  }).catch(error => {
  this.error = error
 })
},
 data() {
  return { currentUser, error: null }
 }
}
</script>
```

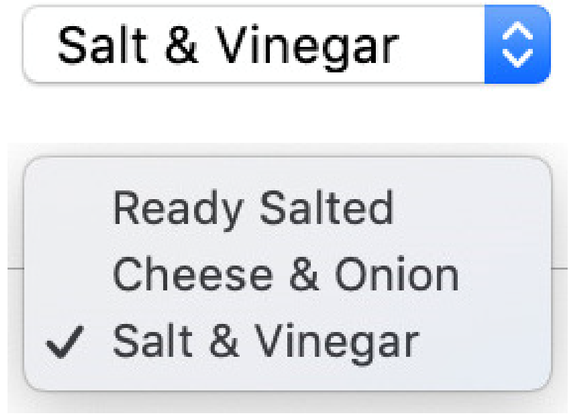
Custom validation of arrays and objects

- Vue allows **custom validators** to be used as props using the **validator property.** This allows us to implement **deep checks** regarding object and collection types.
- Let's look at a CustomSelect component. On a basic level, the prop interface for *select* comprises an array of *options* and a *selected* option.
- Each option should have a label that represents what is displayed in select, and a value representing its actual value.
- The selected option's value can be empty or be equal to the value field for one of our options

```
<template>
 <select>
 <option :selected="selected === o.value"
  v-for="o in options" :key="o.value">
   {{ o.label }}
 </option>
 </select>
</template>

<script>
export default {
 props: {
  selected: {
   type: String
  },
  options: {
  type: Array
  }
 }
}
</script>
```

```
<template>
 <div id="app">
  <CustomSelect :selected="selected" :options="options"/>
 </div>
</template>

<script>
import CustomSelect from './components/CustomSelect.vue'
export default {
 components: {
  CustomSelect
 },
 data() {
  return {
   selected: 'salt-vinegar',
   options: [
    {
    value: 'ready-salted',
    label: 'Ready Salted'
    },
    {
    value: 'cheese-onion',
    label: 'Cheese & Onion'
    },
    {
    value: 'salt-vinegar',
    label: 'Salt & Vinegar'
    },
   ]
  }
 }
}
</script>
```

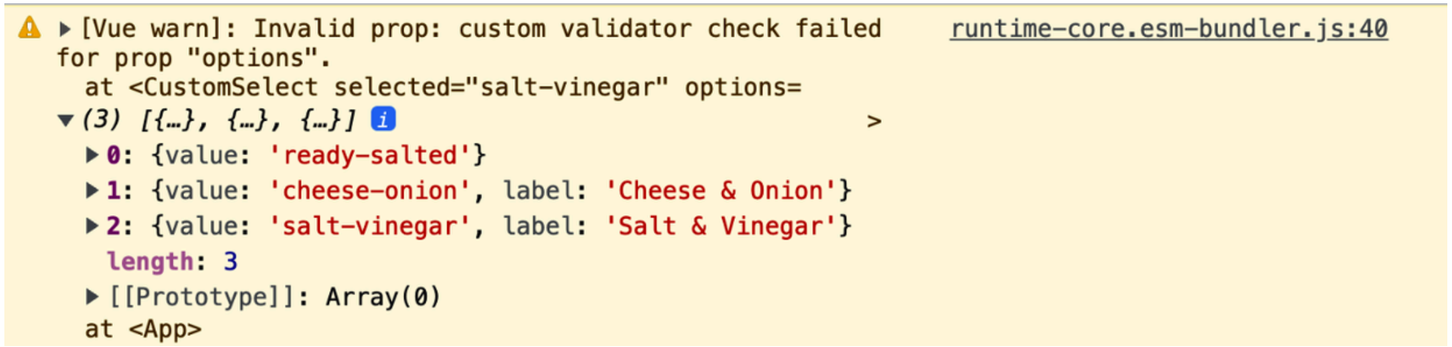
Then use CustomSelect to display a list of British crisp flavors (in src/App.vue):

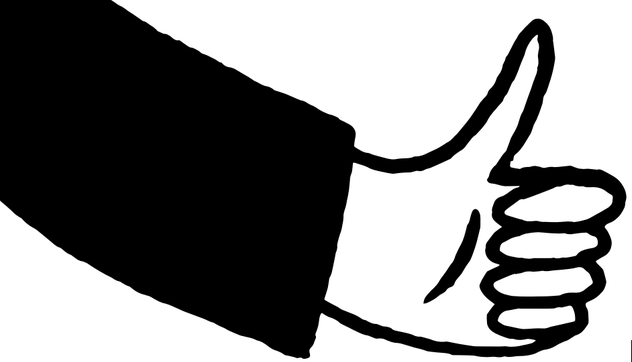


Now we can implement a prop validator method to enable further validation for our component's logic.

```
<script>
export default {
 // other component properties
 props: {
 // other prop definitions
  options: {
   type: Array,
   validator(options) {
   return options.every(o => Boolean(o.value && o.label))
   }
  }
 }
}
</script>
```

If we pass an option with a missing value or label, we will get the following message in the console





Understanding required props

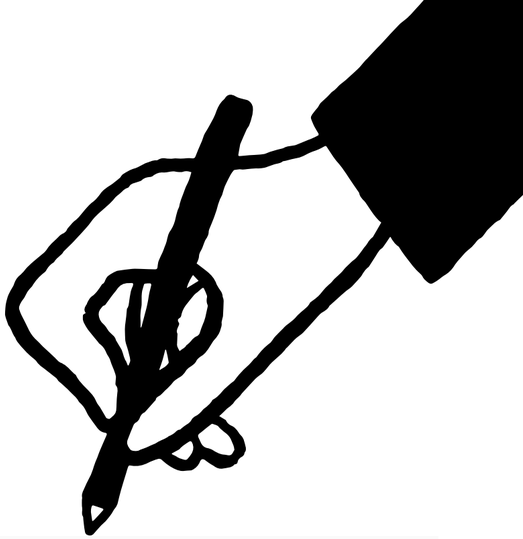
In the *CustomSelect* example, we can make selected a required prop by adding **required:true** to its prop definition

```
<script>
export default {// other component properties
 props: {
  selected: {
   type: String,
   required: true
  }
  // other prop definitions
 }
}
</script>
```

if we don’t pass a value to the selected prop of CustomSelect on the parent component, we will see the following error:

```
⚠ ▶ [Vue warn]: Missing required prop: "selected"
    at <CustomSelect options= ▶ (3) [{…}, {…}, {…}] >
    at <App>
```

Setting the default props value

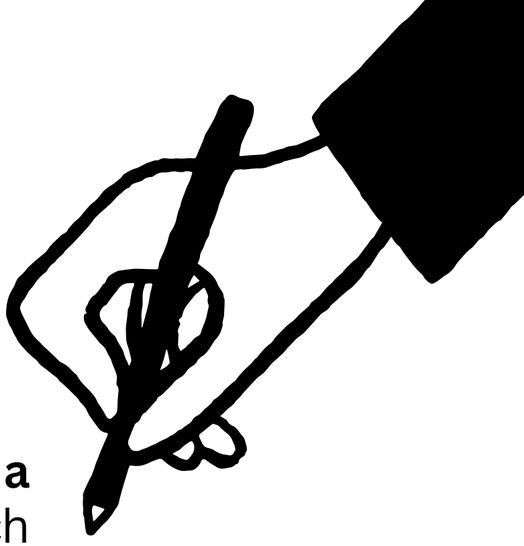


There are situations where setting the **default value for a prop** is good practice to follow.
Take a *PaginatedList* component, for instance. This component takes a **list of items**, a **limit number** of items to display, and an **offset number**.
Then it will display a subset of items – **currentWindow** – based on the limit and offset values:

```
<template>
<ul>
 <li v-for="el in currentWindow" :key="el.id">
  {{ el.content }}
 </li>
</ul>
</template>
```

```
<script>
export default {
 props: {
  items: {
   type: Array,
   required: true,
  },
  limit: {
   type: Number
  },
  offset: {
   type: Number
  }
 },
 computed: {
  currentWindow() {
   return this.items.slice(this.offset, this.limit)
  }
 }
}
</script>
```

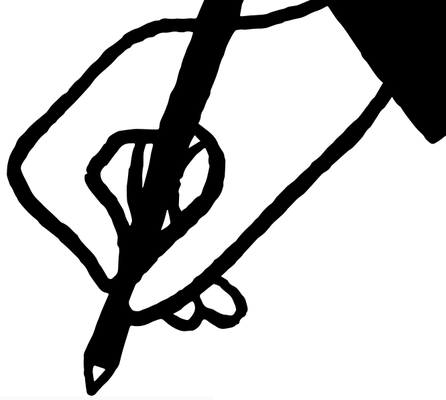
Setting the default props value



Instead of passing the values of limit and offset every time, it might be better to **set limit to a default value** (like 2) and **offset** to 0 (this means that by default, we show the first page, which contains 2 results).

```
<script>
export default {
 props: {
 // other props
  limit: {
   type: Number,
   default: 2,
  },
  offset: {
   type: Number,
   default: 0,
  }
 },
 // other component properties
}
</script>
```

Setting the default props value



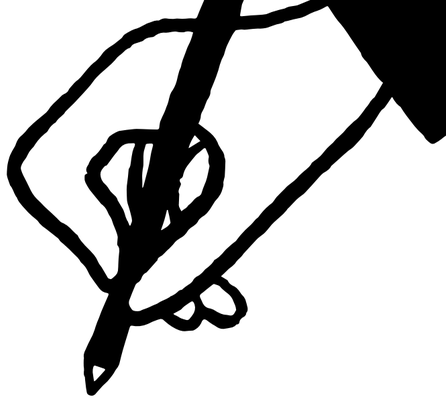
Then in App.vue, we can use *PaginatedList* without passing limit and offset. Vue automatically falls back to the default values in case no value is passed

```
<template>
 <main>
  <PaginatedList :items="snacks" />
 </main>
</template>
```

- Ready Salted
- Cheese & Onion

```
<script>
import PaginatedList from './components/PaginatedList.vue'
export default {
 components: {
  PaginatedList
 },
 data() {
  return {
   snacks: [
   {
    id: 'ready-salted',
    content: 'Ready Salted'
   },
   {
    id: 'cheese-onion',
    content: 'Cheese & Onion'
   },
   {
    id: 'salt-vinegar',
    content: 'Salt & Vinegar'
   },
  ]
  }
 }
}
</script>
```

Setting the default props value

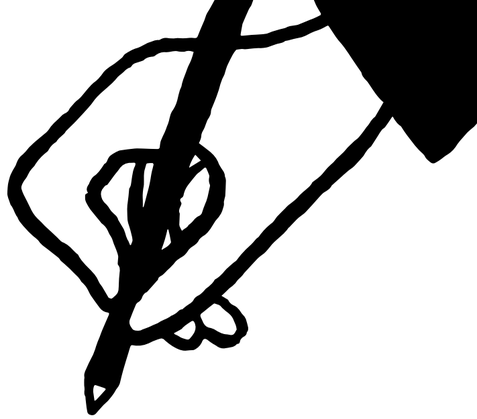


In cases where a prop is an **array** or an **object**, we can't assign its default value with a static array or object. Instead, we need to assign it a **function** that returns the desired default value. We can set the default value of items from the PaginatedList component to an **empty** array:

```
<script>
export default {
 props: {
  items: {
   type: Array,
   default() {
    return []
   }
  }
  // other props
 },
 // other component properties
}
</script>
```

Note here that once you set a default value, you don't need to set required field anymore.

Registering props in <script setup> (setup hook)



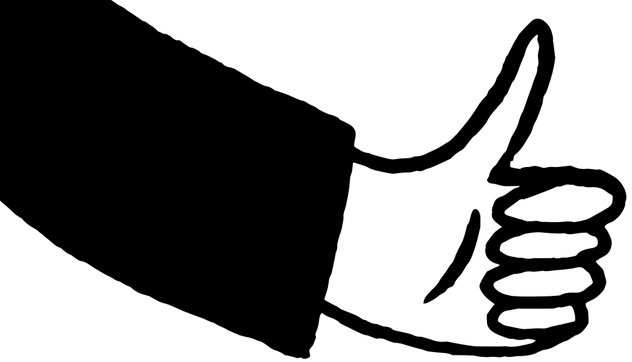
If you use **<script setup>**, since there is no options object, we can't define the component's props using the props field.

Instead, we use the **defineProps()** function from the *Vue* package and pass all the relevant props' definitions to it, just as we did with the props field.

For example, in the **MessageEditor** component, we can rewrite the event registering with **defineEmits**()

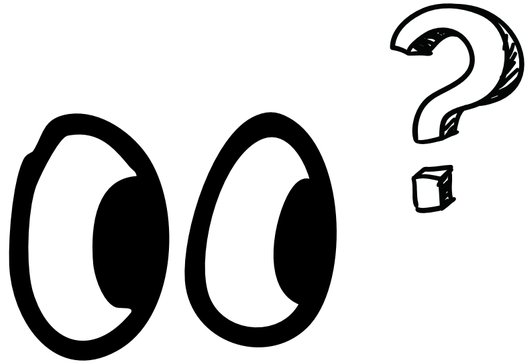
defineProps() returns an object containing all the props' values. We can then access a prop such as items using **props.items** instead within the **script** section, and items as usual in the **template** section. In the previous example, we also use **computed()** to declare a reactive data currentWindow for this component.

```
<script setup>
import { defineProps, computed } from 'vue'
const props = defineProps({
 items: {
  type: Array,
  required: true,
 },
 limit: {
  type: Number
 },
 offset: {
  type: Number
 }
 });
const currentWindow = computed(() => {
 return props.items.slice(props.offset, props.limit)
})
</script>
```



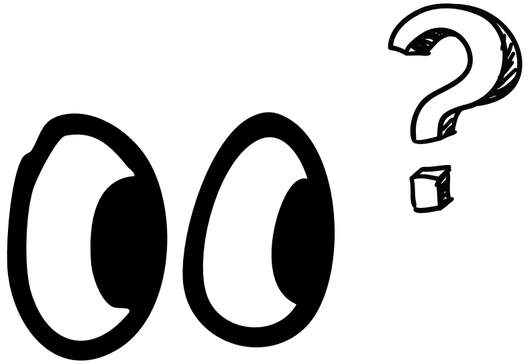
Exercise 4.03 – Validating an Object property

Understanding slots, named slots, and scoped slots



Slots are sections of a component where the template/rendering is delegated back to the parent of the component.
We can consider slots as **templates** or markup that are passed **from a parent to a child** for rendering in its main template.

Understanding slots, named slots, and scoped slots



Passing markup to a component for rendering

The simplest type of slot is the default child slot.
We can define a **Box component** with a slot as follows:

```
<template>
 <div>
  <slot>Slot's placeholder</slot>
 </div>
</template>
```

Behind the scenes, Vue compiles the template section of Box and replaces <slot /> with the content wrapped inside <Box /> from the parent (App). The scope of the replaced content, however, stays within the parent's scope.

The following markup is for the **parent** component (src/**App**.vue):

```
<template>
 <div>
  <Box>
   <h3>This whole h3 is rendered in the slot</h3>
  </Box>
 </div>
</template>
```

```
<script>
import Box from './components/Box.vue'
 export default {
  components: {
   Box
  }
 }
</script>
```

Another example

```
<template>
 <div>
  <Box>
  <h3>This whole h3 is rendered in the slot with parent
      count {{ count }}</h3>
  </Box>
  <button @click="count++">Increment</button>
 </div>
</template>
```

```
<script>
import Box from './components/Box.vue'
export default {
 components: {
  Box
  },
 data() {
  return { count: 0 }
 }
}
</script>
```

The preceding code will render count per its value in the parent component. It does not have access to the Box instance data or props and will generate the following output:

This whole h3 is rendered in the slot with parent count 0

Increment

Incrementing the count from the parent updates the template content, since the count variable in the template passed to Box was bound to data on the parent. This will generate the following output:

This whole h3 is rendered in the slot with parent count 5

Increment

Slots are a way to let the parent have control over rendering a section of a child's template. Any references to instance properties, data, or methods will use the parent component instance. This type of slot does not have access to the child component's properties, props, or data.

Questions?